

X86 结构微程序控制模型研究

Research on the Microprogramming for X86 Processors

马卓* 吴虎成 张民选

Ma Zhuo Wu Hucheng Zhang Minxuan

(国防科技大学计算机学院 长沙 410073)

(School of Computer Science, National University of Defense Technology)

摘 要: 以 Intel®X86 结构为代表的 CISC 微处理器在桌面处理器领域雄踞巨大的市场份额, 因此对 CISC 的研究仍然具有巨大的现实意义。微程序控制是 CISC 结构的一个显著特点, 本文深入剖析了 CISC 处理器中的微程序模型, 并以 Intel®X86 结构为例进行了详细的分析和阐述。

关键字: 复杂指令集计算机; 微程序控制

CISC 结构的持续生命力

从上世纪 90 年代开始, RISC 结构的发展进入了突飞猛进的阶段, 通常的观点认为“CISC 结构中 80% 的指令仅占有 20% 的 CPU 时间, 为其增加的大量硬件资源实质上得不到高效的利用, 造成资源的浪费, 同时也不利于大规模器件的设计实现”^[1]。一时间, 以 RISC 取代 CISC 的浪潮在业界轰轰烈烈的展开, 包括 IBM、DEC、Motorola、MTI 在内的众多业界巨头和研究机构纷纷研制出其完全意义上的 RISC 处理器。但是, 时至今日, 在整个处理器市场内, 占据绝对份额的桌面型处理器仍然是以 Intel 的 X86 系列及其衍生型号为代表的 CISC 处理器。CISC 经久不衰, 一方面可以被认为是 Intel 有效市场运作的产物, 而另一方面也说明了 CISC 并非如先前所宣称的已经走到了末路, 它仍然有着持续而强劲的生命力。

功能强大且长度不定的复杂指令是 CISC 结构指令集的典型代表。在指令功能的界定上, CISC 与 RISC 走了两条完全相反的路线, RISC 结构提供最基本的、使用频度非常高的指令, 面向的目标在于单条指令执行的高效化, 而 CISC 则提供大量面向高级语言的, 功能强大而复杂的指令, 设计的目标在于贴近高级语言的实现。因此, 一般来讲, 复杂指令是 CISC 结构特点的重要体现。复杂指令通常都为多周期指令, 使用常规的译码逻辑进行指令译码将不可避免的导致指令译码逻辑的极度复杂化, 不利于 VLSI 设计实现。Maurice Wilkes 创造性的提出了微程序控制的原理^[2], 极大的简化了译码逻辑, 成为了被目前多数 CISC 处理器采用的通用模型。

本文以 Intel 公司 X86 结构微处理器为例, 重点分析了 X86 微处理器的微程序控制模型, 对复杂指令的微程序转换、微程序的组织、中断和异常的处理、测试与自测试等方面展开了系统的研究, 详细说明了作为 CISC 核心部件的微程序模块, 为我国研制具有自主知识产权并与 Intel 的 X86 指令集兼容的微处理器做出贡献。

复杂指令的微程序构成

提出微程序控制的主要目的在于对 CISC 结构复杂指令进行可靠译码。通常, 在 CISC 结构中, 为了更加贴近高级语言实现, 复杂指令绝大多数都是功能强大的多周期指令, 对这些指令的直接译码无疑需要使用大量逻辑电路, 并且这些逻辑电路将呈现极度复杂化。因此, 在 CISC 结构中, 复杂指令的译码通常由微程序来承担。根据 Wilkes 的理论, 任何一条指令都可以分解为若干微指令的组合。

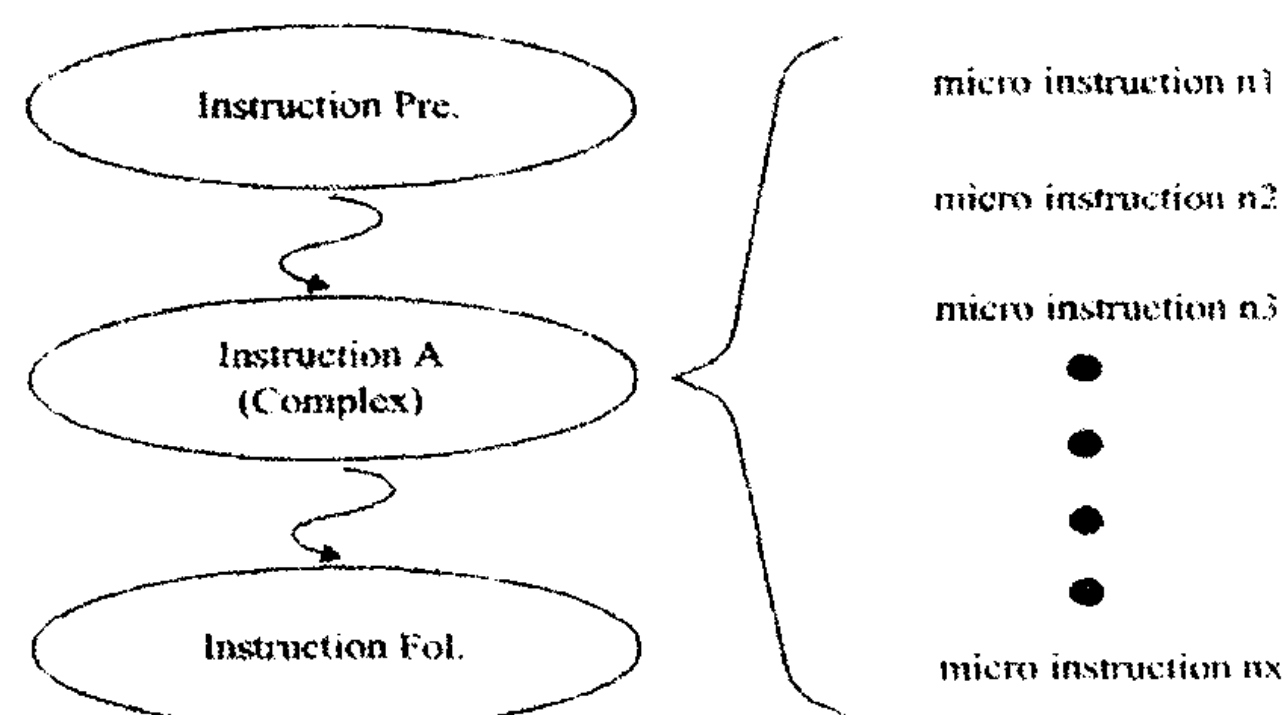


图 1 复杂指令的分解

* 本文受国家自然科学基金项目资助, 项目编号 60376018

作者简介: 马卓, 男, 国防科技大学计算机学院微电子研究所研究实习员, 研究方向为微处理器设计
通信地址: 410073 湖南长沙国防科技大学计算机学院。

图 1 给出了复杂指令与微指令的对等关系。指令序列中的复杂指令 A 可以分解为 X 条微指令的有序组合。与直接逻辑译码相比，使用微程序分解复杂指令的主要优点在于能使用简单的逻辑实现微操作信号的生成，避免使用复杂的逻辑电路和庞大的状态机控制。

表 1 微程序转换与直接逻辑译码各项指标对比

性能指标	微程序转换	直接逻辑译码
逻辑复杂程度	较简单	复杂
状态机的使用	没有状态机或仅有简单状态机	需要复杂的状态机控制
占用的芯片面积	适中	巨大
功耗	较小	很大
包含错误的可能性	较为不易包含设计错误	非常容易引入设计错误
速度	较慢	很快
专利保护的性能	容易保护	不容易保护

表 1 给出了微程序译码模型与直接逻辑译码模型在复杂程度、功耗、面积等几个方面的对比情况。可见，在 CISC 结构中，因为大量功能复杂的指令的存在，使用微程序译码模型能更好的解决由于指令功能复杂带来的功耗、面积、设计复杂度等问题。虽然使用微程序译码模型会带来译码速度下降的问题，但是对于执行时间很长的复杂指令来说，微程序译码的方式是一种较好的折衷选择。因此，结合微程序译码和直接逻辑译码的特点，在现代 CISC 微处理器中，通常将两者结合使用，将简单指令交由直接逻辑译码处理，而复杂指令则由微程序部件实现微操作信号的生成。本文在下面的篇幅中将重点讨论微程序相关的问题。

1.1 微程序的垂直组织

从逻辑上来讲，微程序与程序一样，都由指令或者微指令构成。一般的，将微指令的指令流组织称为微程序的垂直组织，而将每条微指令中微操作的组织方式称为微程序的水平组织^[3]。

微程序作为指令的一种细粒度解释，在数量上会大大超过复杂指令，因此，微程序的垂直组织成为了微程序设计中的重点问题。

图 2 表示了 X86 结构中微程序的垂直组织情况。整个微程序空间由 M 条微指令构成，物理上顺序存放，存在 n 个微程序入口。通常，在微程序的垂直组织上存在两种方法，一种是将一段完整的微程序顺序组织在一起，这种方法不需要插入额外的跳转指令，微程序显得更为简洁，

但是由此带来的是不同的微程序段的入口地址不存在简单的规律性，无疑是增加了入口地址生成逻辑的复杂度；另一种方法是以规整段的方式组织微程序，每 L 条微指令构成一个基本段，在一个基本段内不能结束的微程序占用多个基本段，前一个基本段的最后是一条具有微跳转功能的指令，通过微跳转将多个基本段链接起来，这种方式无疑会增加不少额外的跳转控制，但是由于微程序的入口显得非常的规律，由此而获得是微程序入口地址生成逻辑的极大简化，因此综合性能在前一种方法之上。在 X86 系列微处理器中使用的是后一种方法。

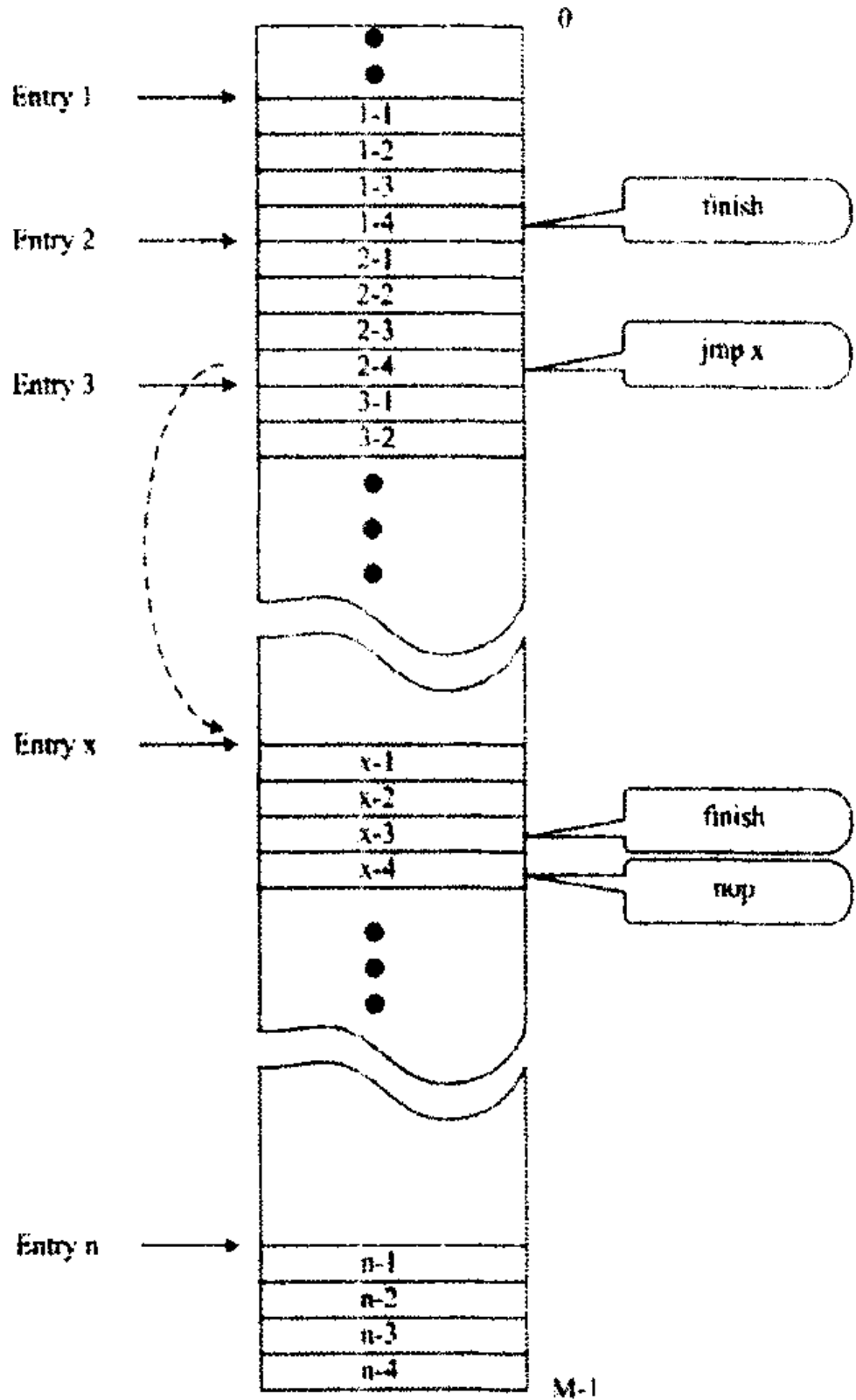


图 2 微程序的垂直组织

图 2 所示，Entry 1 入口处的微程序由四条微指令构成，正好存放于一个基本段内，1-4 处的微指令具有微程序结束的标识；Entry 2 入口处的微程序由七条微指令构成，因此需要两个基本块来存放，根据规整段式组织的原则，2-4 处的微指令具有跳转标识至 Entry x 入口，并在 x-4 处填写空操作指令。这就是 X86 结构中采用的垂直组织微程序的方式，微程序的入口固定的存在于模 4 为 0 的入口点上，极大的减化了入口地址成生逻辑。

1.2 微程序的水平组织

一般来讲，组成微程序的基本单位是微指令，而微指令是由微操作构成的，处理器内部存在大量的微控制信号，因此微指令的长度大大超过指令的长度。通常，在微指令的水平组织上也存在两种基本的方法，一种是微指令的每个位对应于处理器内部的一个微控制信号，这种方法使

用直接控制，不需要额外的译码逻辑，控制信号传递迅速，显而易见，这种方法适用于微操作数量较少的微处理器；另一种方法是将微控制信号分组并编码，编码后的结果作为微指令的不同的域从而组成微指令，这种方法需要对微指令的不同的域分别译码才能生成最终的控制信号，信号传递较慢，但是这种方法能有效减少微指令的长度，因而适用于具有大量微控制信号微处理器。

图 3 给出了 X86 结构中微指令的水平组织情况。微指令由六个基本域组成，总长 90 位，其表示如下：

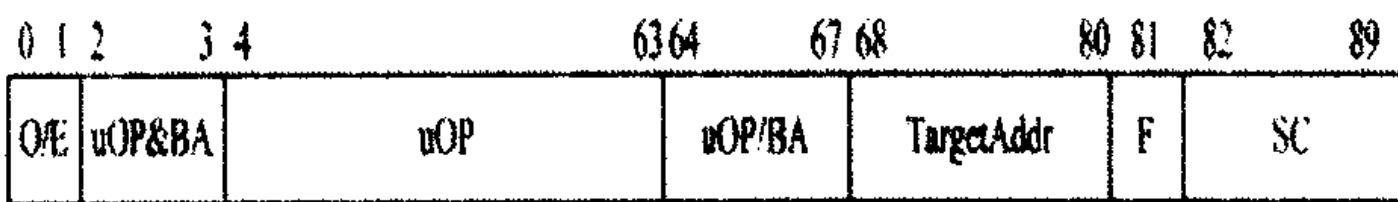


图 3 微程序的水平组织

- O/E：奇偶校验位；
- uOP&BA：微控制信号、微程序流控制域，这里微控制信号是针对微程序部件的，因此可以认为是与微程序流控制信号复用的；
- uOP：微控制信号域，针对不同功能部件的微控制信号成组的方式组织于微指令的 uOP 域；
- uOP/BA：微控制信号、微程序流控制域，根据不同的指令两者复用；
- TagerAddr：微程序自跳转目标地址；
- F：微程序结束标志；
- SC：微程序部件的状态及跳转控制域，控制微程序流的方向。

由于 X86 微处理器中具有数量庞大的微控制信号，微指令中的微控制信号需要经各个功能部件的译码逻辑进行简单的译码之后才能作为不同部件的控制信号。

1.3 微程序的错误检测

微程序所产生的微控制信号直接或者间接的控制功能部件，需要使用专门的信号传递通道，因此，在绝大多数的设计中，微程序都是以 ROM 为载体直接存放于处理器内部的微程序部件之中，微操作以 ROM 点阵的形式予以固化，并被称之为码点。码点的正确性与随即逻辑的正确性一样决定处理器的正确性。因此，如何检测码点的正确性成为微程序部件设计中的一个重要问题。

X86 结构微处理器中使用了两种方式来检测码点的正确性。一种是随码点一同编码的奇偶校验，另一种是对全部码点阵列的自检测响应压缩分析。

- 奇偶校验：在§1.2中介绍了在微程序的水平组织上存在奇偶校验位，每条码点的读出都经过奇偶校验；

- 自检测响应压缩校验：奇偶校验仅仅具有有限的检测错误的能力，而响应压缩算法则能从更大的程度上保证码点的正确性。

X86 结构使用的响应压缩模型如图 4 所示，响应压缩采用的特征多项式为：

$$(1 + x^{85} + x^{87} + x^{88} + x^{89})$$

使用该特征多项式对 ROM 的全部码点遍历之后，将产生的结果与预定的结果比较，由此产生响应压缩检测的报错信息。

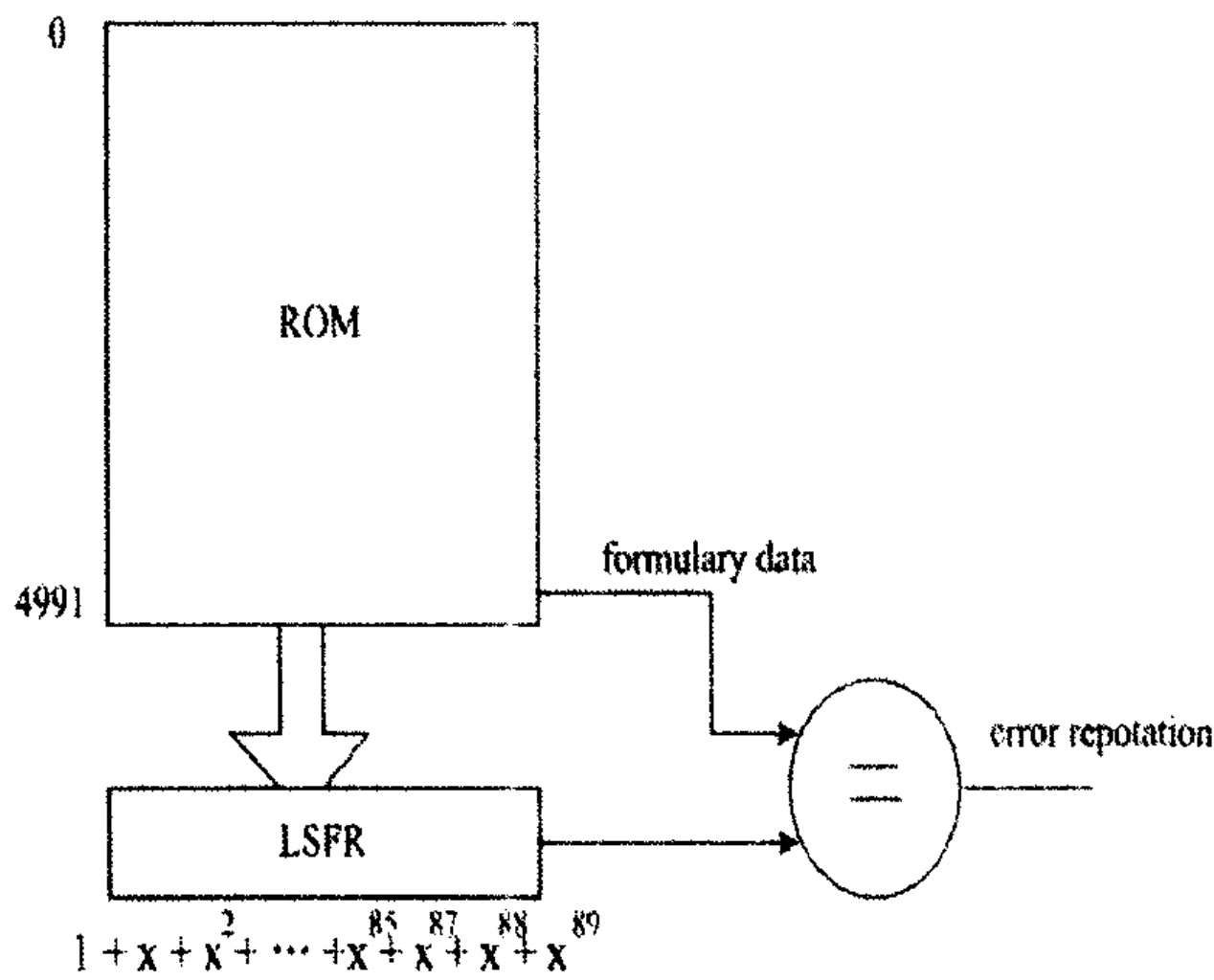


图 4 响应压缩的结构

微程序访问地址的生成

上文提到过，微程序模型相比直接逻辑译码最大的缺点在于控制信号的生成速度。为了能够提高微程序译码的执行速度，高效的微程序访问地址的生成逻辑是必要的。X86 结构微处理器中，微程序采用基本段的存放方式，并且集成了中断/故障处理、测试等等功能，因此，微程序访问地址的生成方式显得较为复杂。

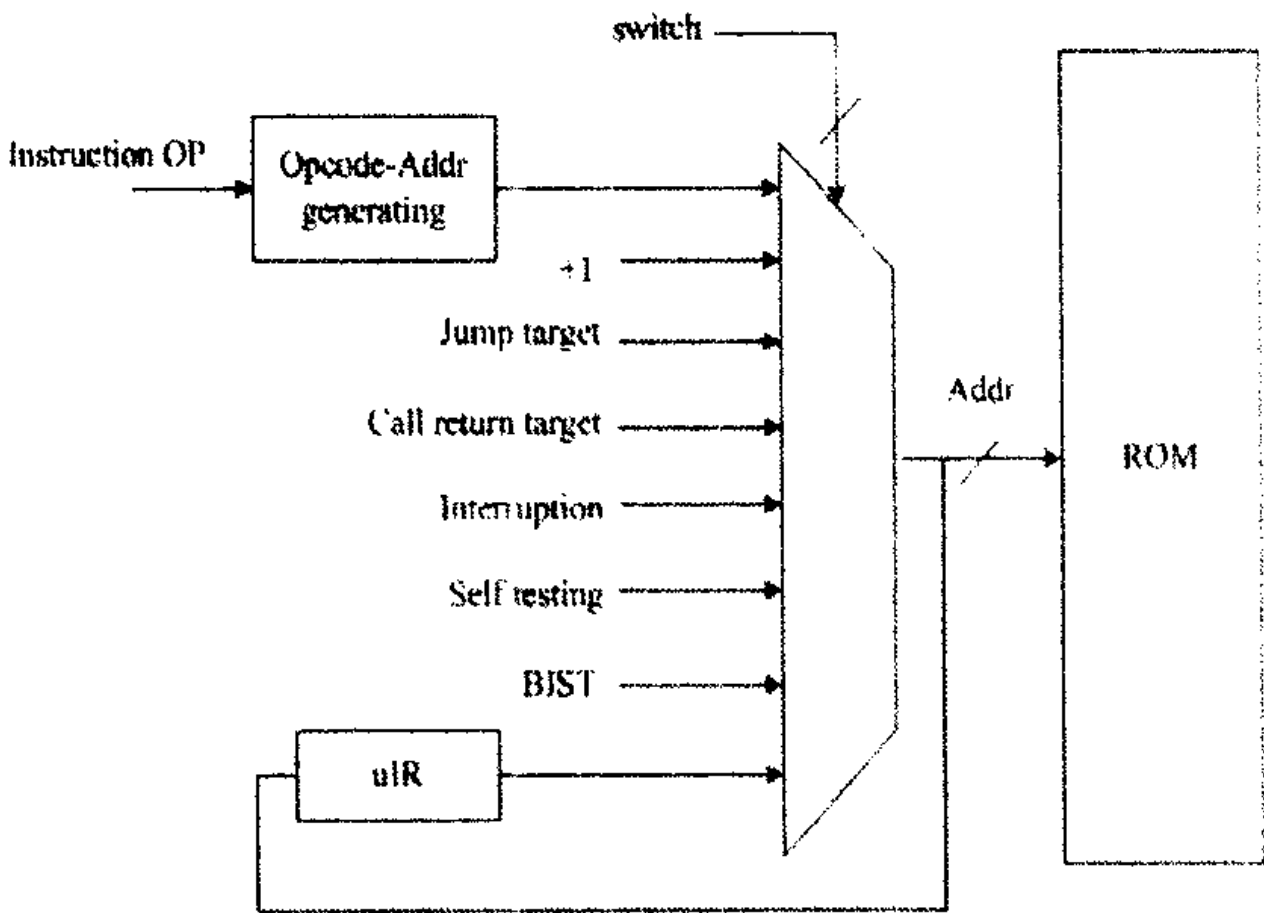
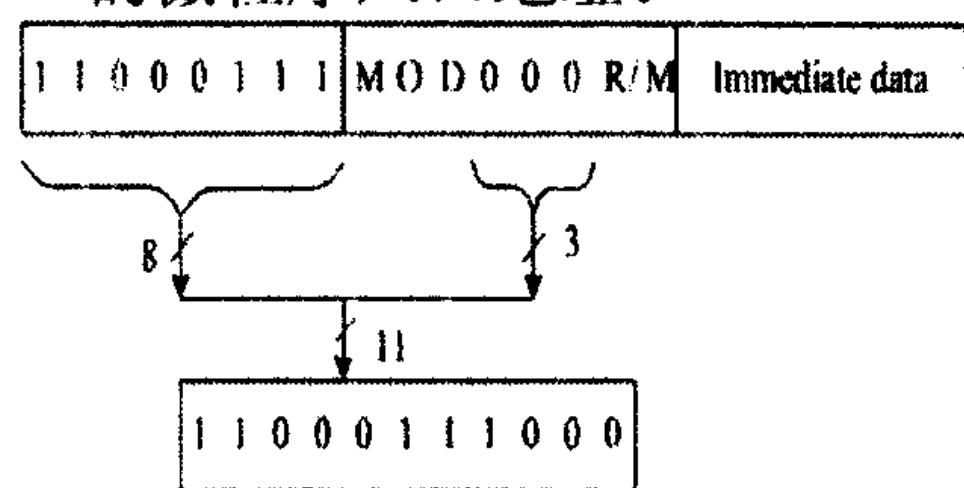


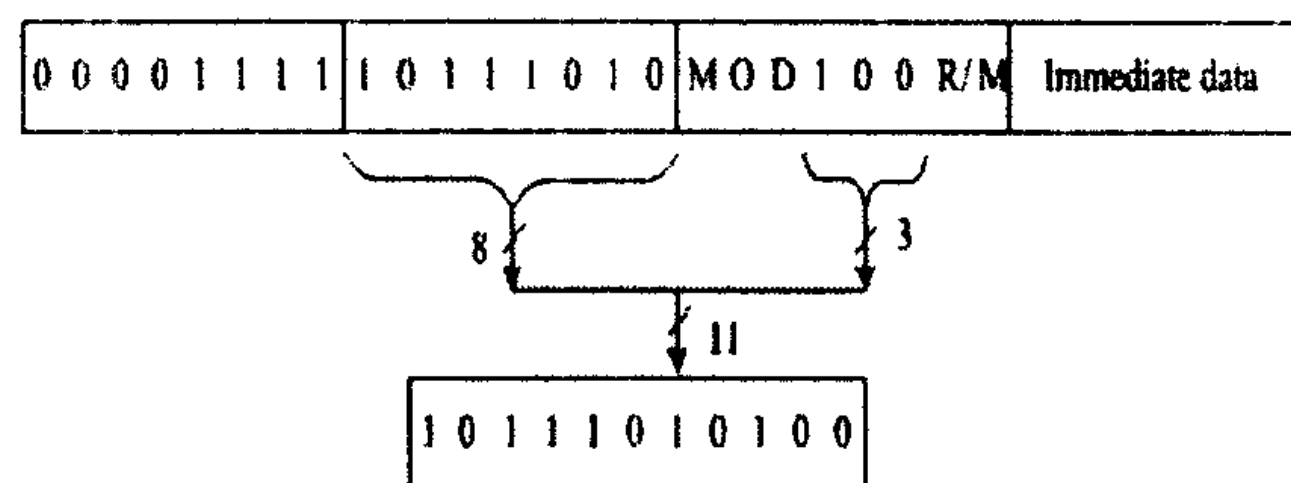
图 5 微程序 ROM 访问地址的生成

图 5 显示了 X86 结构中微程序访问地址的生成逻辑框图，微程序 ROM 地址的来源包括指令入口、中断/故障入口、自测试入口、测试入口、子程序调用返回目标、跳转目标、顺序地址、当前暂停地址等八类。

- 指令入口地址：微程序部件最主要的功能就是完成复杂指令的译码，因此，指令入口成为了微程序 ROM 最主要的入口之一。指令入口地址的生成应该由指令编码中提取有效部分来生成，通常使用的是指令的操作码和访存方式等信息，由于 X86 结构中存在单/双字节操作码的两类指令，因此对于操作码和访存方式等信息的提取应该区别对待。图 6 给出了针对这两类指令的信息提取方式。提取出来的 11 位信息经位扩展，生成 13 位的微程序入口地址。



A) 单字节操作码地址生成信息位提取



B) 双字节操作码地址生成信息位提取

图 6 操作码地址生成信息的提取方式

- 中断/故障入口地址：中断/故障是微程序部件的扩展功能，使用微程序处理中断/故障具有设计简单高效的特点。中断故障的入口由中断事件类型经排队编码后给出。
- 自测试入口：自测试入口即微程序码点响应压缩自测试的起始地址。
- 测试入口：X86 结构的微处理器中，微程序部件集成了对主要功能部件进行测试的程序代码，通过测试指令或者边界扫描启动功能部件的测试，因此必须设置测试功能的入口。
- 子程序调用返回目标地址：与常规程序类似，微程序也存在子程序的调用问题，将子程序调用微指令的顺序地址通过本入口加入，实现可靠的子程序调用返回。
- 跳转的目标地址：微程序的自跳转目标，即微指令码中的 TargetAddr 域。
- 顺序地址：即自增 1 地址，用于指示微指令的顺序取指。
- 当前暂停地址：执行复杂指令时若出现流水线暂停，则需要保持当前微指令地址。

微程序访问地址的生成方式是微程序部件功能的重要体现。X86 结构中，微程序部件根据

指令/微指令的执行状态，从上述八路地址中选择一路作为进一步取指的目标地址，有效的强化了微程序部件的功能。

微程序部件的附加功能

一般的说，微程序是主要用于复杂指令译码处理并固化与处理器内部的，这种结构使得设计者可以除复杂指令译码之外的更多功能以微程序的形式集成在微程序部件之中。在 X86 系列微处理器中，设计者将包括中断/故障处理、芯片测试等功能集成到了微程序部件里，使得 X86 的微程序部件具有了更为强大的功能。

1.4 中断/故障处理

对于同步的中断/故障，X86 结构是采用微程序来实现处理的，相关的中断服务处理以微程序的形式固化在微程序部件之中。

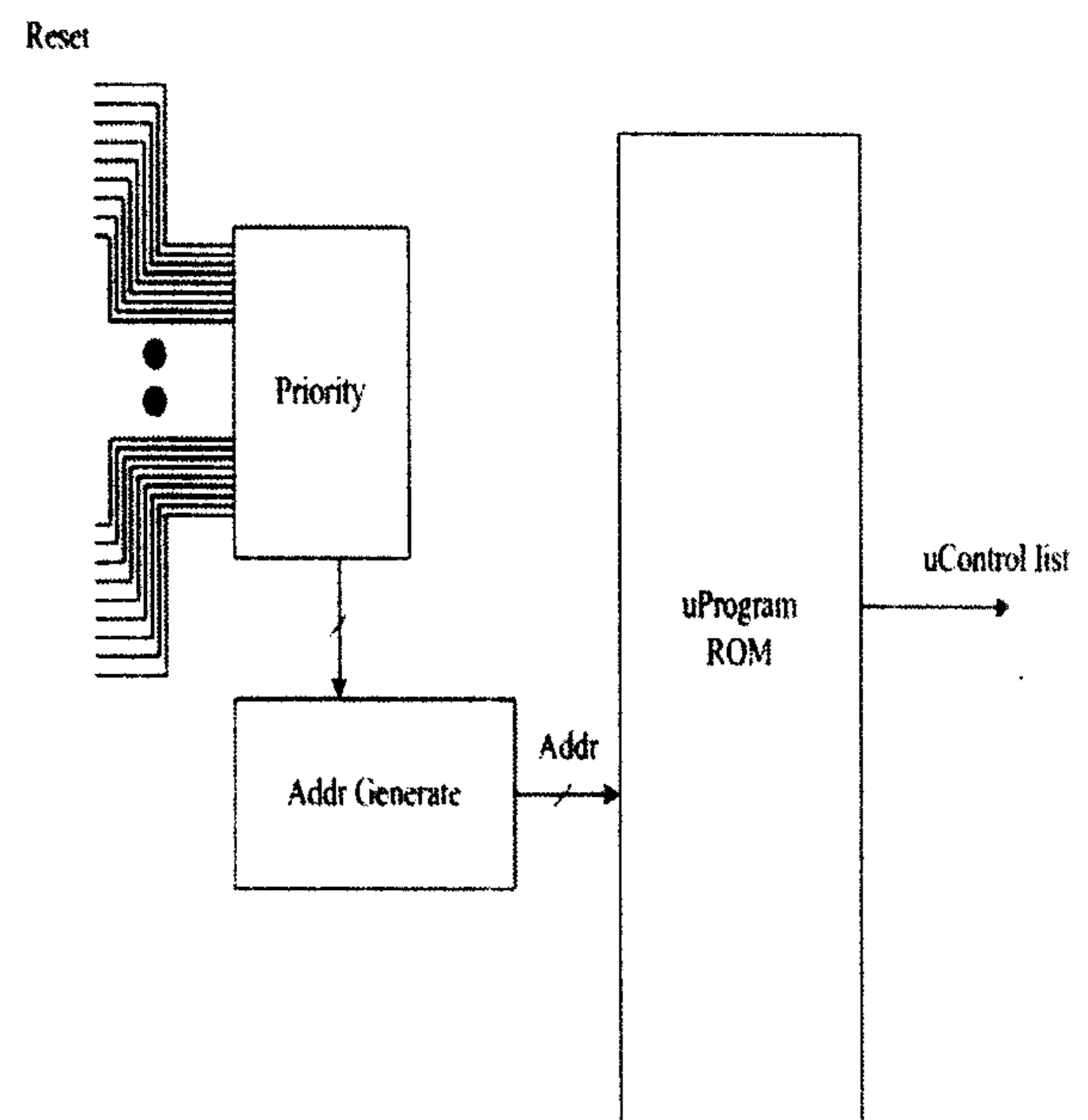


图 7 微程序部件的中断处理结构

图 7 是微程序部件中中断/故障处理的结构，所有同步的中断事件经优先级排序之后，送往微程序 ROM 的地址生成逻辑，产生中断/故障的入口地址。特别值得一提的是，在 X86 结构中，复位过程也是以中断的形式给出的。一般来说，复位是一种异步中断，X86 结构将复位作为优先级最高的中断事件，通过简单的逻辑，利用复位信号固定的生成入口地址 38H。程序的执行转入复位入口以后，由微程序部件引导处理器执行复位过程。处理器的复位过程包括很多操作，例如状态/控制寄存器的初值填写，浮点部件的初始化，段寄存器地址寄存器的初始化，初始访存地址的生成等工作。

其它由微程序部件负责处理的中断事件也遵循上述的过程。

1.5 芯片测试功能

微程序除了能完成中断/故障的处理之外，X86 结构中还集成了部分的芯片测试功能。通过外围引脚的配置，X86 处理器可以在初始化过程中对芯片内部功能部件进行测试和诊断。

一般来讲，芯片内部的测试分为存储器测试和随机逻辑测试两大部分。对于存储器的测试，X86 处理器采用的是经典的“写入—读出—比较”的测试方式。

图 8 是内部存储器微程序测试的基本框图。微程序对全部内部存储器空间进行 0-1 特征值的写入，读出后与特征值进行比较，从而判断存储器是否工作正常。

相对来说，对随机逻辑的测试要复杂一些。构建随机逻辑测试的微程序之前，首先要确定需要进行测试的随机逻辑占全部逻辑的比例，在 X86 处理器中，这一比例高达 70%（包括上文说明的存储器测试）。随机逻辑的测试主要集中在功能组件（运算部件类）、控制组件（指控类）、通路三个部分。通过定义三个部分测试的优先级，以功能组件最高通路最低，可以基于已经测试的部分来进行低优先级组件的测试。

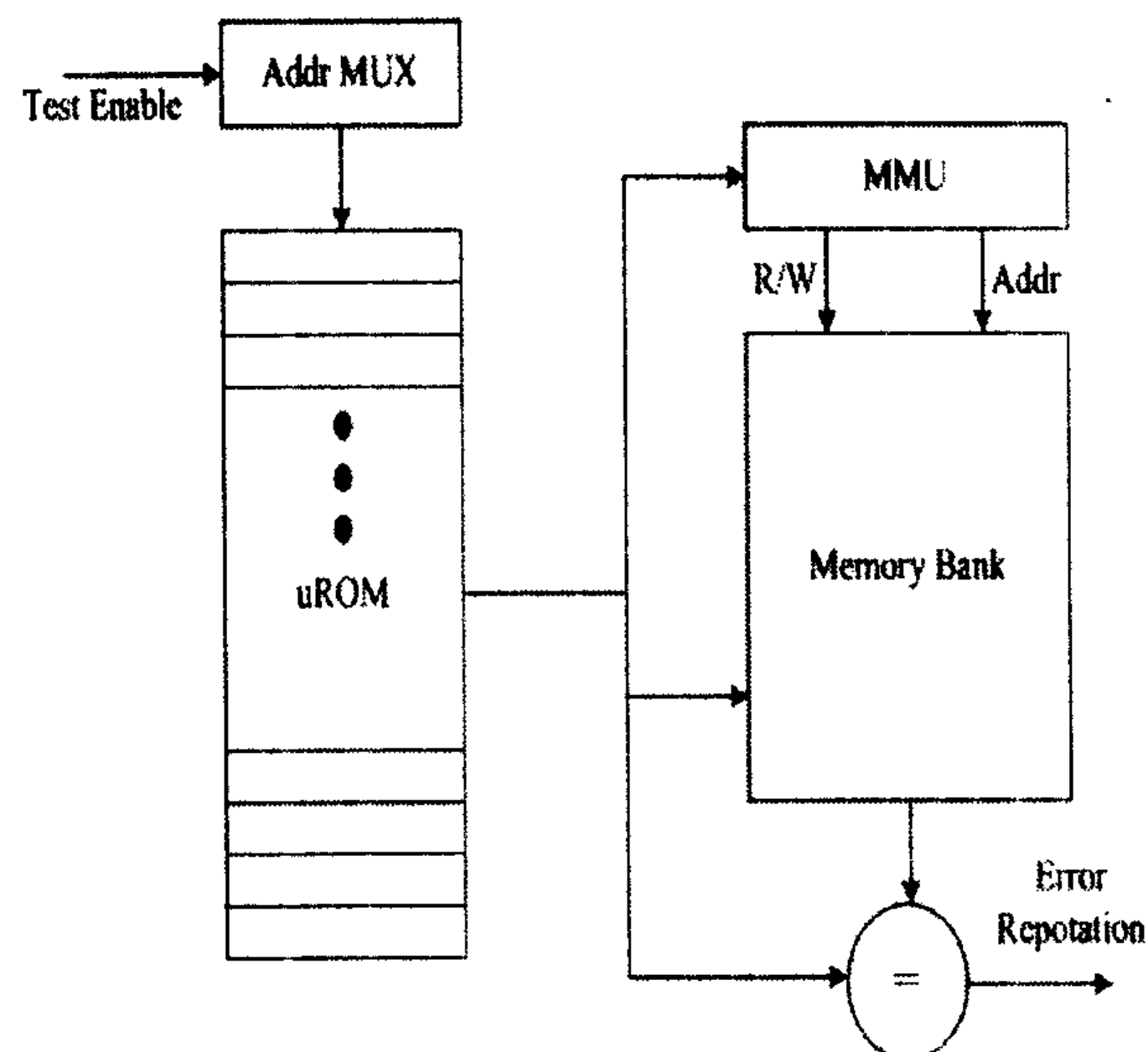


图 8 片内存储器的微程序测试

功能组件的测试：由于微指令与指令的执行效果是等价的，因此，可以构建一个微程序段，采用

“执行—比较”的方式进行功能组件的测试。

- 图 9 给出了以 ALU 为例的功能组件测试结构。

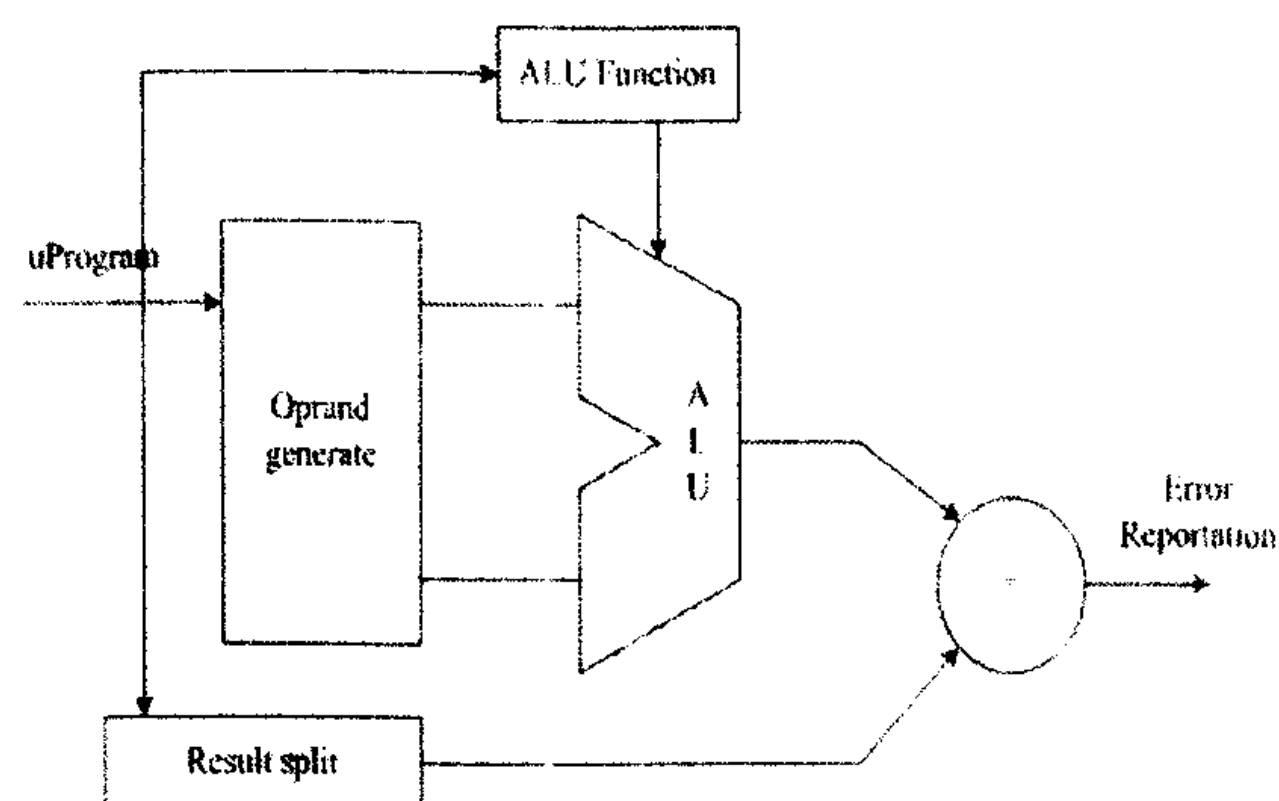


图 9 以 ALU 为例的功能组件测试

- 控制组件的测试：以简单指令译码逻辑为例，译码逻辑的测试集中在控制信号他通路的测试上。因为优先级较高的功能组件的测试已经完成，所以可以通过增加简单指令序列来实现测试功能，并通过对不同的执行部件的执行结果的收集比较，可以实现对错误的定位。
- 通路的测试：通路的测试主要是对众多的旁路信号的测试。基于前面高优先级组件正确的信息，采用类似的方式就可以实现对通路逻辑的测试。

由于工艺的稳定程度等原因，芯片的测试成为确保芯片正常工作的必要手段。X86 处理器通过在微程序部件中集成芯片的测试功能，使得在芯片启动工作之初就能有效的对内部逻辑/存储器进行测试，简化了芯片的测试过程。

结束语

X86 结构作为一款经典的 CISC 处理器结构，是目前大部分计算机系统的核心，而微程序控制又是 X86 结构最显著的特点。本文对 X86 结构中微程序控制中微程序组织、访问地址生成、微程序部件附加功能等几个问题展开的研究，有利于对 Intel 微处理器技术的理解，有利于我国自主设计与 Intel X86 结构兼容的微处理器芯片。

参考文献

- 【1】冯晓兵，马湘宁，“二进制翻译和动态优化技术”，中国科学院计算机数研究所信息技术快报 2003-6 期
- 【2】M. V. Wilkes, “The Best Way to Design an Automatic Calculating Machine”, Proc. Manchester Univ. Computer Inaugural Conf., Ferranti Ltd., 1951, pp. 16-18
- 【3】Intel Architecture Software Developer's Manual, Volume 1: Basic Architecture
- 【4】Intel Architecture Software Developer's Manual, Volume 2: Instruction Set Reference
- 【5】Intel Architecture Software Developer's Manual, Volume 3: System Programming